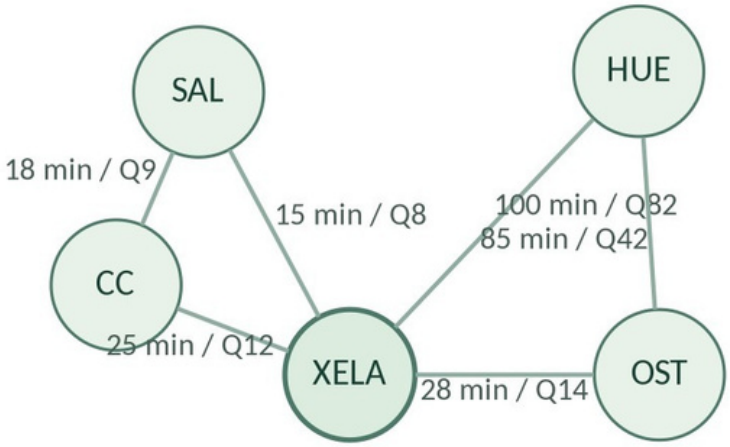
ESTRUCTURA DE DATOS II

Manual técnico



TRANSRED

Estructuras de datos, algoritmos e integración real del sistema

1. Qué resuelve TransRed y cómo está organizado

TransRed es una aplicación de escritorio que administra paradas y conexiones de transporte del suroccidente de Guatemala que nos permite registrar y buscar paradas, establecer trayectos con un tiempo y costo asociado, encontrar recorridos según uno de esos criterios y consultar si las paradas pertenecen a una misma componente conectada y el escenario se representa como un conjunto de paradas y rutas, no como un servicio de tráfico en tiempo real. Los tiempos y costos proceden del archivo de datos; la visualización del mapa es esquemática a partir de coordenadas.

CAPA / PROYECTO	RESPONSABILIDAD COMPROBADA
TransRed.Core	Modelos, TablaHash, Trie, Grafo, UnionFind, GestorTransporte y persistencia JSON.
TransRed.Interfaz	Aplicación WinForms con Explorar red, Paradas, Conexiones y Algoritmos.
TransRed.Consola	Menú alternativo para registrar, buscar y consultar las estructuras.
Datos/transred.json	Persistencia de las paradas y conexiones de la red.

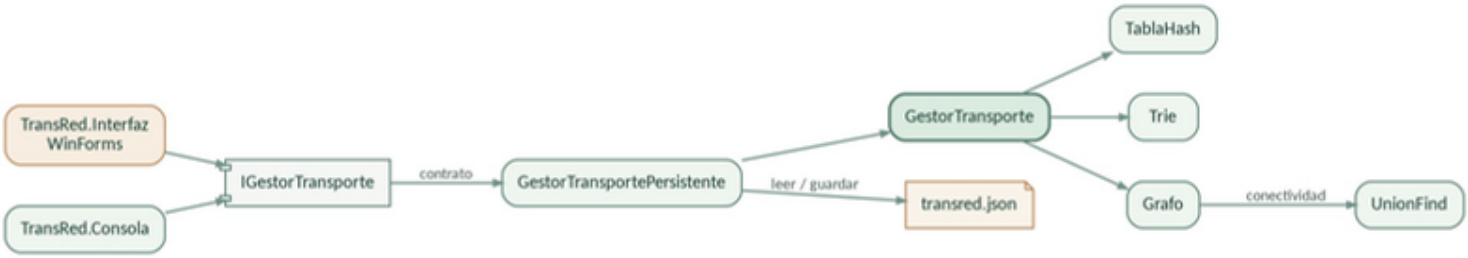


Figura 1. Relaciones lógicas verificadas entre aplicaciones, gestor, estructuras y almacenamiento.

Las dos aplicaciones usan el contrato IGestorTransporte. El gestor persistente delega operaciones al gestor central y guarda el estado cuando corresponde. Así, las reglas de las estructuras permanecen fuera de los formularios.

2. Tabla Hash: búsqueda por código

2.1. Por qué se usa

Cada parada cuenta con un código único, como XELA, SMA u OST, y buscarla recorriendo todas las paradas sería posible, pero obliga a revisar más elementos conforme crece la red. La Tabla Hash transforma el código en una posición de un arreglo, por eso resulta adecuada para consultas frecuentes por identificador.

2.2.Cómo está implementada

La clase TablaHash posee un arreglo fijo de 101 cubetas y cada cubeta guarda un nodo enlazado con una Parada y una referencia Siguiente y si dos códigos caen en la misma posición, ambos se conservan en una lista; no se sobrescribe el registro anterior.

CalcularIndice recorre los caracteres del código en mayúsculas. En cada paso multiplica el valor acumulado por 31 y suma el carácter, finalmente obtiene el módulo de 101.

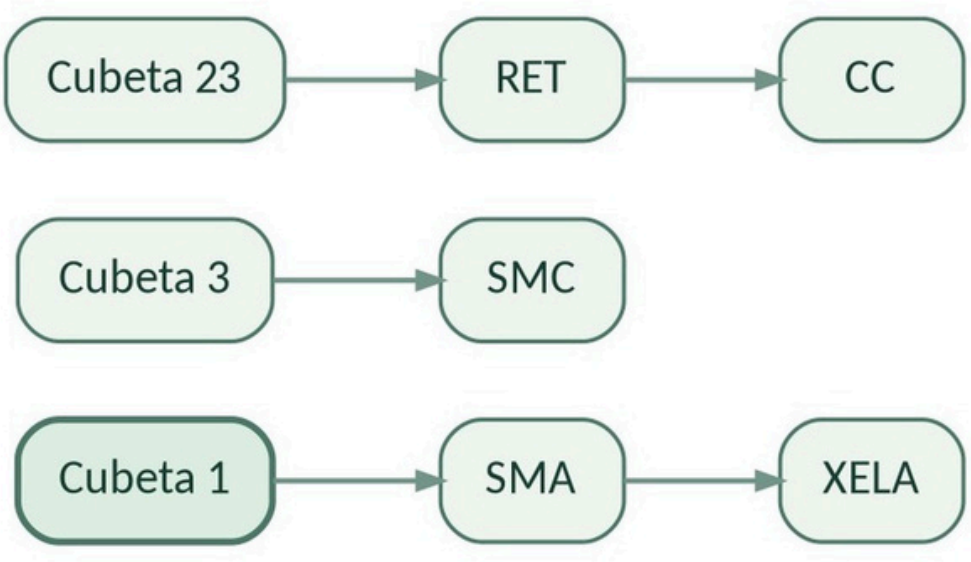


Figura 2. Cubetas y colisiones reales del conjunto actual: XELA/SMA en 1, CC/RET en 23.

Ejemplo real: XELA y SMA producen el índice 1, y al insertar el nodo nuevo se coloca al inicio de la lista, por lo tanto, cuando ambos están cargados, la cubeta puede recorrerse como SMA → XELA. La búsqueda revisa los códigos de esa cubeta hasta encontrar la coincidencia.

MÉTODO	OPERACIÓN	DECISIÓN
Insertar	Calcula la cubeta y recorre la lista antes de insertar.	Impide códigos repetidos, sin requerir otra estructura.
Buscar	Recorre la cadena de la cubeta calculada.	Compara códigos sin distinguir mayúsculas/minúsculas.
Eliminar	Desenlaza el nodo de la lista correspondiente.	Conserva el resto de elementos que colisionaron.
ObtenerTodas	Recorre cada cubeta y cada cadena.	Permite llenar tablas y sincronizar datos.

2.3.Ventaja y sacrificio

En condiciones razonables de distribución, la búsqueda es corta. El sacrificio es disponer de 101 posiciones aunque existan menos paradas y aceptar que, con muchas colisiones, una búsqueda se vuelva lineal dentro de la cubeta. El código actual no reajusta automáticamente la capacidad.

3. Trie: sugerencias por nombre

3.1. Por qué complementa a la Hash

La Tabla Hash responde bien si el usuario conoce un código exacto, y el Trie resuelve otra necesidad, que es escribir el inicio del nombre y recibir sugerencias sin recorrer todas las paradas en busca de coincidencias de texto. Se utiliza para consultar prefijos de nombres, no para calcular rutas.

3.2. Representación real

La clase Trie guarda un nodo raíz y, para cada carácter, nodos con PrimerHijo y SiguienteHermano. También conserva una referencia Parada en el nodo donde termina un nombre completo. La entrada se normaliza, no se eliminan tildes ni se busca por palabras interiores.

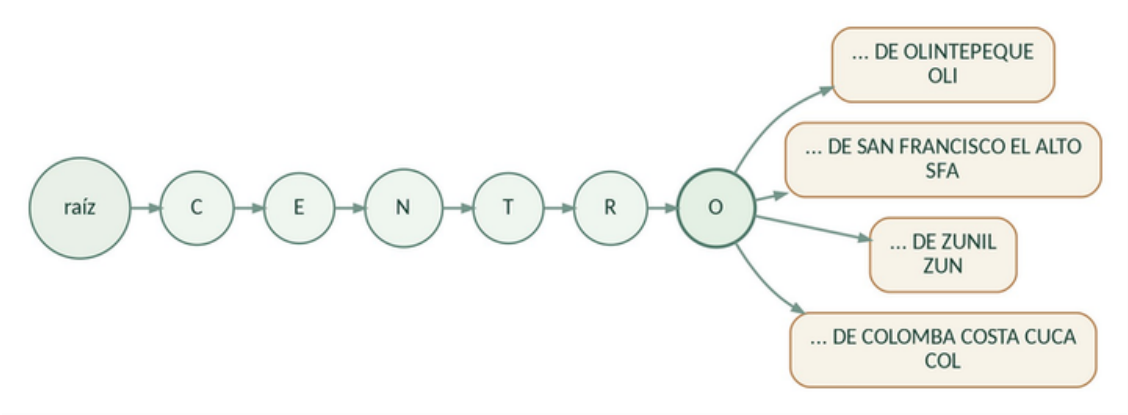


Figura 3. Prefijo real CENTRO y cuatro destinos encontrados; se compactan visualmente los sufijos.

Si se consulta el prefijo “Centro”, el método BuscarPorPrefijo avanza por los caracteres C-E-N-T-R-O y reúne los nombres que continúan desde ese nodo, con los datos actuales corresponde a OLI, SFA, ZUN y COL, el gráfico resume los sufijos para mostrar el principio

4. Grafo: paradas, conexiones y pesos

El grafo es el centro del sistema de rutas, cada Parada corresponde a un vértice y cada conexión se representa como una arista que apunta a otro vértice, las aristas guardan dos valores independientes: Minutos y Costo, así la misma red puede responder preguntas distintas sin alterar sus conexiones.

4.1. Representación utilizada

Grafo mantiene un arreglo dinámico de vértices, vértice referencia una lista enlazada de aristas a través de PrimeraArista, las aristas guardan el índice del destino, minutos, costo y un enlace Siguiente, el arreglo crece duplicando su capacidad cuando se llena.

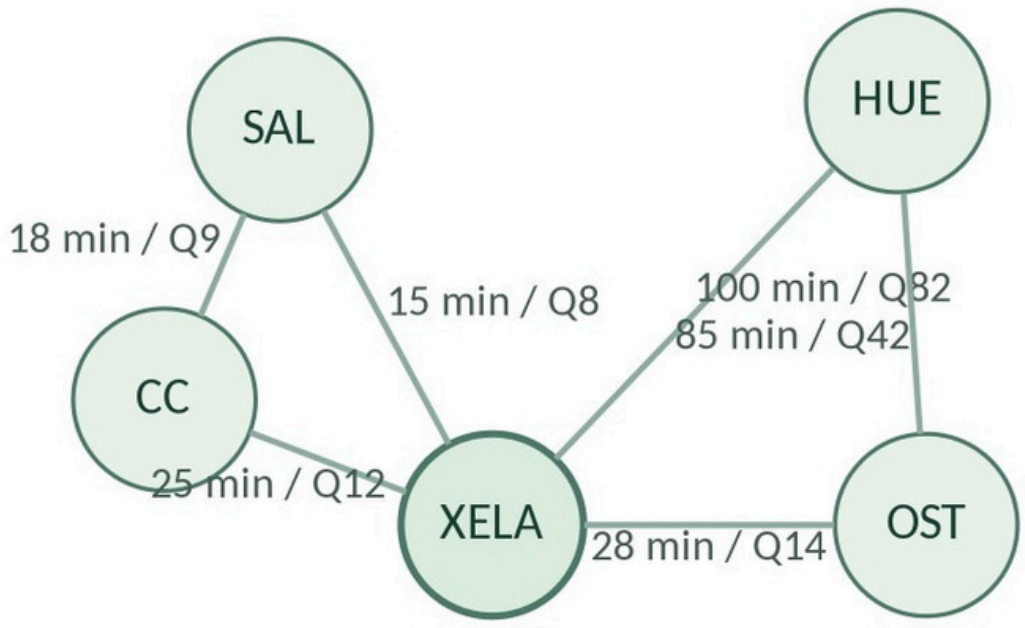


Figura 4. Subgrafo real con pesos de tiempo y costo tomados de transred.json.

Los trayectos pueden registrarse como dirigidos o bidireccionales. En el segundo caso se insertan dos aristas, una por cada sentido. El archivo de prueba actual contiene 102 aristas dirigidas que corresponden a 51 pares en ambos sentidos; ello no convierte a la clase Grafo en una estructura exclusivamente no dirigida.

4.2. Qué operaciones permite

OPERACIÓN	QUÉ SUCEDE
AgregarParada	Se agrega un vértice si su código aún no existe.
RegistrarRuta	Se validan extremos, pesos no negativos y aristas duplicadas; se inserta una o dos aristas.
EliminarRuta	Se retira la conexión del sentido solicitado y, opcionalmente, la inversa.
EliminarParada	Se retiran las aristas asociadas y se ajustan índices internos del arreglo.
ObtenerConexiones	Se convierten las aristas internas en objetos Conexion para la interfaz o JSON.

5. BFS y DFS: dos maneras de recorrer

5.1.BFS —búsqueda en anchura

BFS comienza en una parada y recorre primero las paradas alcanzadas a menor número de saltos, en la implementación, un nodo queda marcado como visitado cuando se en cola.

5.2.DFS— búsqueda en profundidad

DFS también parte de una parada, pero emplea una pila implementada mediante un arreglo int[] y un índice de cima. Extrae el último vértice agregado e incorpora vecinos aún no visitados. El código marca los nodos en el momento de apilarlos; por ello, su orden exacto depende del orden de las listas de adyacencia y de esta elección de marcado.

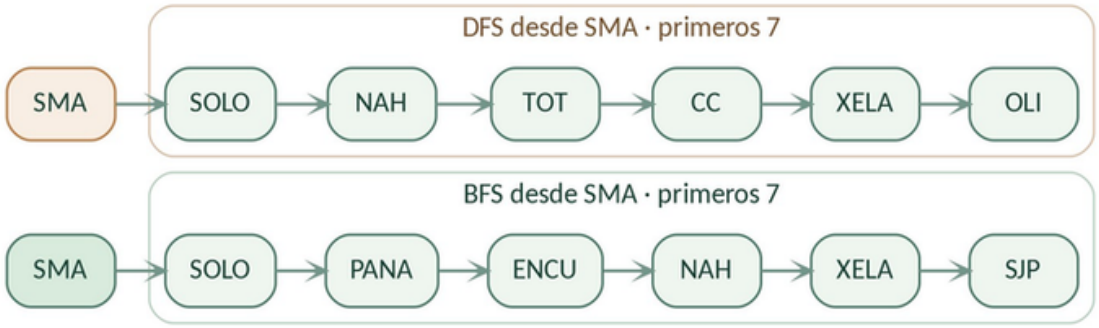


Figura 5. Primeras siete visitas derivadas del orden del código y datos actuales, iniciando en SMA.

PUNTO	BFS	DFS
Estructura de apoyo	Cola (Queue<int>)	Pila sobre int[]
Tendencia	Explora por niveles	Profundiza según el orden de inserción
Valor práctico	Visualizar alcance y vecindad	Explorar ramas y conectividad
Salida real	Arreglo de Parada visitadas	Arreglo de Parada visitadas

Ambos recorridos visitan los vértices alcanzables; no calculan, por sí mismos, la ruta más rápida o barata. En la pantalla Algoritmos se ilumina el orden que devuelve el Core. La animación reproduce ese resultado; no expone la cola ni la pila internas paso a paso.

6. Dijkstra: elegir por tiempo o por costo

La aplicación permite dos criterios de ruta sobre el mismo grafo. Si el usuario elige “Más rápida”, el peso utilizado es Minutos. Si elige “Económica”, el peso utilizado es Costo, y en ambos casos, el método ObtenerResultadoRuta usa una variante de Dijkstra: mantiene distancias, predecesores y un registro de vértices visitados, y selecciona por búsqueda lineal el siguiente vértice con menor distancia acumulada.

6.1. Ejemplo que muestra la diferencia

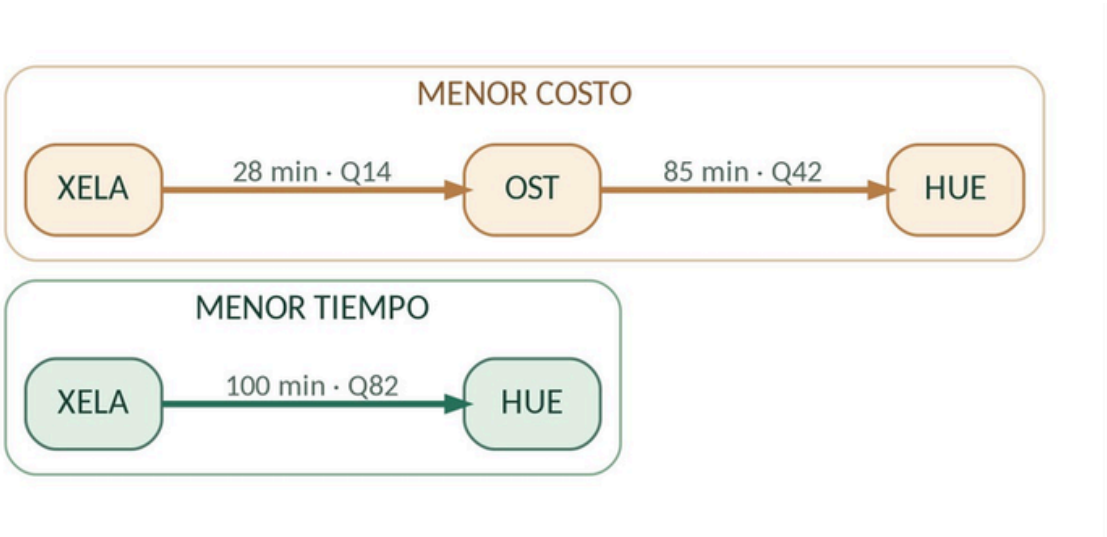


Figura 6. XELA → HUE: el criterio de tiempo y el de costo conducen a rutas diferentes.

CRITERIO	RECORRIDO	TIEMPO TOTAL	COSTO TOTAL
Más rápida	XELA → HUE	100 min	Q82
Más económica	XELA → OST → HUE	113 min	Q56

En la ruta directa XELA → HUE el costo es Q82. La alternativa XELA → OST → HUE suma Q14 + Q42 = Q56, pero necesita 28 + 85 = 113 minutos. Es un ejemplo concreto de por qué no existe una única “mejor ruta” sin definir antes el criterio.

6.2. Qué devuelve el algoritmo

El resultado es un objeto ResultadoRuta con el arreglo ordenado de paradas, TiempoTotal y CostoTotal, y la interfaz utiliza esos valores para mostrar el resumen y resaltar el trayecto en el mapa, el mapa no participa en la decisión: únicamente representa la solución calculada en el Core.

6.3.Condiciones y límites

Dijkstra requiere pesos no negativosm RegistrarRuta rechaza tiempos o costos negativos y valores NaN o infinitos. Si no hay camino hacia el destino, ObtenerResultadoRuta devuelve null; si un código no existe también devuelve null. Su implementación actual no usa una cola de prioridad: recorre linealmente los vértices para seleccionar el mínimo. Es una opción simple para 35 paradas, aunque escalaría peor con redes mucho mayores.

7. Union-Find: comprobar conectividad

Union-Find agrupa vértices que pertenecen a la misma componente conectad, acá se mantiene dos arreglos: padres, que indican el representante de cada conjunto, y rangos, que ayudan a unir árboles sin hacerlos innecesariamente profundos. Encontrar utiliza compresión de caminos; Unir utiliza unión por rango.

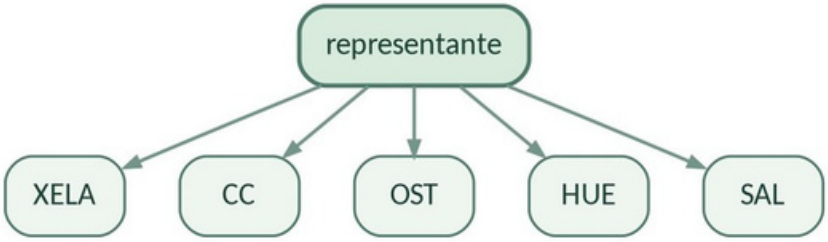


Figura 7. Ilustración conceptual de un conjunto con representante común, no es una captura del arreglo interno real.

7.1.Cómo se usa realmente en TransRed

El método Grafo.EstanConectadas crea un UnionFind nuevo en cada consulta, recorre las aristas y une los índices de sus extremos, y luego compara los representantes del origen y el destino, esto significa que no conserva permanentemente una estructura de conjuntos actualizada entre operaciones.

Se eligió por su claridad para responder una pregunta de componentes. Su principal sacrificio dentro del programa es reconstruir los conjuntos al consultar, en lugar de mantenerlos incrementalmente. Para probar un caso negativo significativo sería necesario trabajar con una copia de datos que contenga una parada aislada.

8. Integración, persistencia y validaciones

8.1. Registro coordinado de una parada

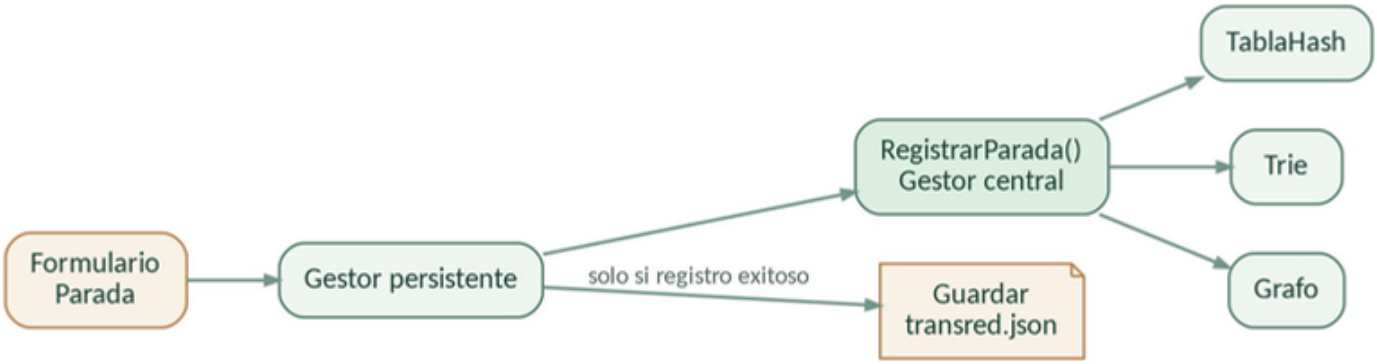


Figura 8. Camino principal de registro, que va al gestor, estructuras y guardado.

RegistrarParada comprueba primero que el código no exista en la Tabla Hash, después inserta la parada en Hash, Trie y Grafo. Si falla la inserción en una estructura posterior, revierte los cambios anteriores realizados por el gestor. Esta coordinación evita dejar un nombre indexado en el Trie sin un registro correspondiente en la Hash cuando se produce uno de esos fallos controlados.

GestorTransportePersistente envuelve las operaciones del gestor, al registrar o eliminar con éxito una parada o una ruta, invoca Guardar para escribir el estado en JSON, las consultas de lectura no disparan guardados.

8.2. Qué se guarda

CAMPO	CONTENIDO
Nombre / Version	Identificación y versión de formato del conjunto de datos.
Paradas[]	Codigo, Nombre, Latitud y Longitud.
Conexiones[]	Origen, Destino, Minutos y Costo por arista dirigida.

El archivo es Datos/transred.json. AlmacenamientoJson lo escribe con sangría y emplea un archivo temporal .tmp antes de sustituir el destino, al iniciar, GestorTransportePersistente carga ese JSON, valida datos y reconstruye el gestor y sus estructuras. Las aristas guardadas se cargan una a una como dirigidas: la bidireccionalidad se representa explícitamente por las dos entradas.

8.3.Interfaz y responsabilidades

La pantalla Explorar red permite seleccionar origen y destino, BuscadorTrie consulta los prefijos de nombre y, si no hay coincidencias, ofrece búsqueda por prefijo de código, el botón calcula la ruta mediante el gestor.

El LienzoRed resalta esa secuencia y proporciona zoom, desplazamiento y selección de nodos. Las pantallas Paradas y Conexiones gestionan registros. Algoritmos ofrece BFS, DFS y Union-Find; su grafo visual utiliza las paradas y conexiones obtenidas del gestor.

Actualmente el menú principal tiene cuatro secciones: Explorar red, Paradas, Conexiones y Algoritmos, la clase VistaPlanificar permanece en el proyecto como reutilización de ExploradorPrincipal, pero no se ofrece como quinta sección para evitar duplicar la funcionalidad.

9. Complejidad y decisiones de diseño

Las siguientes cifras describen las operaciones principales del código. No son mediciones de tiempo realizadas en un equipo, sino que son análisis razonados a partir de sus ciclos, listas y arreglos. V significa vértices, E aristas, L longitud de código o prefijo, k tamaño de una cadena de colisiones y α la inversa de Ackermann.

ESTRUCTURA / OPERACIÓN	COSTO ORIENTATIVO	LECTURA PRÁCTICA
Hash: buscar código	O(L + k)	Calcular índice y recorrer solo la cadena afectada.
Trie	Depende de L, hermanos y resultados	Cada carácter puede recorrer hermanos; recopilar hace copias de arreglo.
BFS	O(V + E)	Cola y recorrido de listas de adyacencia.
DFS	O(V + E)	Pila y recorrido de listas de adyacencia.
Dijkstra	$O(V^2 + E)$	Selecciona el mínimo mediante búsquedas lineales.
Union-Find: una consulta completa	O(V + E·α(V))	Construye conjuntos nuevos sobre todas las aristas.
Guardar JSON	O(V + E), sin contar E/S	Serializa el estado completo tras cada cambio exitoso.

9.1. Qué se gana y qué se sacrifica

DECISIÓN	SE GANA	SE SACRIFICA
Hash con 101 cubetas	Consulta por código sencilla y rápida en una red pequeña.	Memoria reservada y cadenas más largas si aumentan colisiones.
Trie de hijos/hermanos	Prefijos compartidos, sin depender de librería de autocompletado.	Más nodos; búsqueda entre hermanos y copias en Recopilar.
Grafo con listas	Representación natural de relaciones y dos pesos por arista.	Buscar índices/aristas puede implicar recorrido lineal.
Dijkstra sin heap	Código más directo y fácil de explicar.	Menor eficiencia en redes de gran tamaño. Trabajo repetido en consultas sucesivas.
Union-Find reconstruido	Sincronía simple con el grafo al consultar.	Escritura del conjunto completo en cada modificación.
Guardar JSON completo	Formato legible y fácil de compartir.	

10. Datos de prueba y plan de evidencias

10.1.Verificación estática del conjunto de datos

Se revisó el archivo transred.json de la versión documentada. Este análisis estático verifica su contenido y coherencia básica; no sustituye las ejecuciones reales de la interfaz o consola.

INDICADOR	RESULTADO OBSERVADO
Paradas declaradas	35; supera el mínimo solicitado de 30.
Aristas dirigidas declaradas	102; equivalen a 51 pares bidireccionales.
Códigos duplicados de parada	No detectados.
Aristas dirigidas duplicadas	No detectadas.
Extremos inexistentes en conexiones	No detectados.
Tiempos/costos negativos o no finitos	No detectados.
Alcance entre paradas con el conjunto actual	35 de 35 alcanzables desde cada origen al recorrer aristas.

10.2.Consultas

OPCIÓN	5 ENTRADAS QUE DEBEN EJECUTARSE
Hash / código	XELA, SMA, CC, OST, HUE
Trie / prefijo	Centro, Parque Central, Acceso, Embarcadero, Cuatro
BFS / inicio	SMA, XELA, CC, HUE, COL
DFS / inicio	SMA, XELA, CC, HUE, COL
Dijkstra / tiempo	XELA→HUE, XELA→OST, XELA→COL, SMA→SMC, CC→SAL
Dijkstra / costo	XELA→HUE, XELA→OST, XELA→COL, SMA→SMC, CC→SAL
Union-Find / pares	XELA ↔ HUE, SMA ↔ SMC, CC ↔ COL, OST ↔ SJP, XELA ↔ TEC

10.3. Resultados de referencia derivados de los datos

CONSULTA	RUTA / RESULTADO CALCULADO	ESTADO
XELA→OST (tiempo)	XELA → OST · 28 min · Q14	Referencia del JSON
XELA→COL (tiempo)	XELA → COL · 65 min · Q30	Referencia del JSON
XELA→HUE (tiempo)	XELA → HUE · 100 min · Q82	Referencia del JSON
XELA→HUE (costo)	XELA → OST → HUE · 113 min · Q56	Referencia del JSON
SMA→SMC (tiempo)	SMA → SOLO → ENCU → CC → XELA → SM → SMC · 193 min · Q95	Referencia del JSON

11. Ejecución, límites y conclusiones

11.1.Cómo ejecutar el proyecto

El sistema está construido en C# y .NET 10. La interfaz usa Windows Forms y, por su configuración net10.0-windows, debe ejecutarse en Windows, la consola está configurada para net10.0. El archivo de solución es Red_Transporte.slnx.

Desde la carpeta Red_Transporte se utilizan los comandos:

dotnet buildRed_Transporte.slnx
dotnet run –project TransRed.Interfaz
dotnet run –project TransRed.Consola

11.2.Límites observables

Para probar registros y eliminaciones debe trabajarse con una copia de Datos/transred.json, porque las modificaciones se guardan. El repositorio contiene los datos de ejemplo listos para revisar.

- La red representa relaciones y pesos definidos en el JSON, no hay cálculo de tráfico ni horarios en vivo.
- El dibujo del mapa es esquemático, calculado a partir de las coordenadas; no es un mapa carretero con geometría de calles.
- Las consultas del Trie se realizan por inicio de nombre normalizado, no por cualquier palabra interior ni por coincidencias tolerantes a tildes.
- La conectividad de Union-Find considera las aristas sin dirección y reconstruye el conjunto en cada consulta.
- El código actual no incluye una opción para editar directamente una parada existente, se registran, buscan y eliminan paradas.

11.3.Conclusión técnica

TransRed combina estructuras que resuelven tareas diferentes. La Hash identifica registros por código, el Trie encuentra nombres por prefijo, el Grafo modela rutas y permite aplicar BFS, DFS y Dijkstra, Union-Find responde consultas de componentes conectadas, el gestor mantiene esas operaciones coordinadas y la persistencia JSON conserva el estado entre ejecuciones.

La decisión principal del proyecto es separar el problema en estructuras con responsabilidades concretas. Para el tamaño de prueba 35 paradas y 102 aristas dirigidas las implementaciones son comprensibles y suficientes para demostrar su funcionamiento. Sus límites también son identificables: búsquedas lineales internas, reconstrucción de Union-Find y escrituras completas del JSON. Reconocerlos permite defender el proyecto con fundamento, no solo mostrar que la interfaz funciona.

Anexo A. Relación entre funciones y archivos

FUNCIONALIDAD	CÓDIGO PRINCIPAL
Buscar/registrar/eliminar parada	Core/Estructuras/TablaHash.cs · Core/Servicios/GestorTransporteServicio.cs
Sugerencias por prefijo	Core/Estructuras/Trie.cs · Interfaz/Components/BuscadorTrie.cs
Paradas, aristas y pesos	Core/Estructuras/Grafo.cs
BFS, DFS, Dijkstra, conectividad	Core/Estructuras/Grafo.cs · Core/Estructuras/UnionFind.cs
Carga/guardado	Core/Servicios/Persistencia/GestorTransportePersistente.cs · AlmacenamientoJson.cs
Mapa animado e itinerario	Interfaz/Components/LienzoRed.cs · Views/ExploradorPrincipal.cs
Visualización BFS/DFS/UF	Interfaz/Views/VistaAlgoritmos.cs
Gestión por formularios	Interfaz/Views/VistaParadas.cs · VistaConexiones.cs

T

Anexo B. Espacios para las capturas de ejecución

EVIDENCIA 01 · POR INSERTAR

ESPACIO RESERVADO PARA CAPTURA REAL

Pantalla Paradas: código consultado, registro localizado y total de paradas.

Figura E1. Búsqueda y registro de parada. Captura pendiente de incorporar.

Anexo B. Espacios para las capturas de ejecución

EVIDENCIA 01

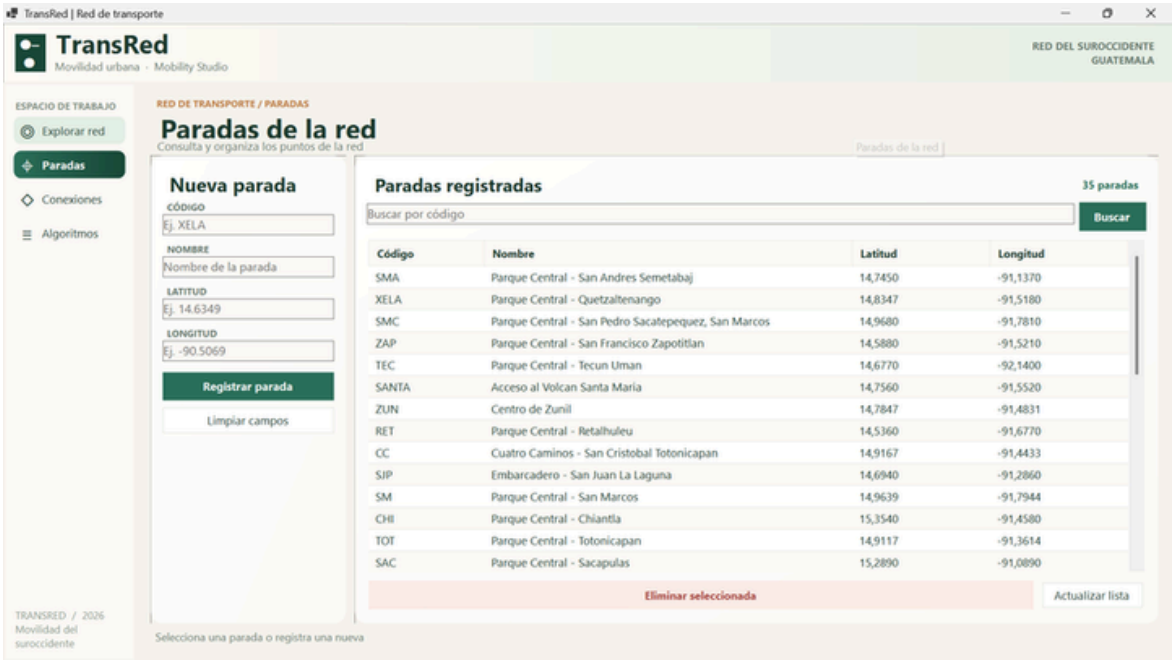


Figura E1. Búsqueda y registro de parada

EVIDENCIA 02



Figura E2. Autocompletado por prefijo.

EVIDENCIA 03

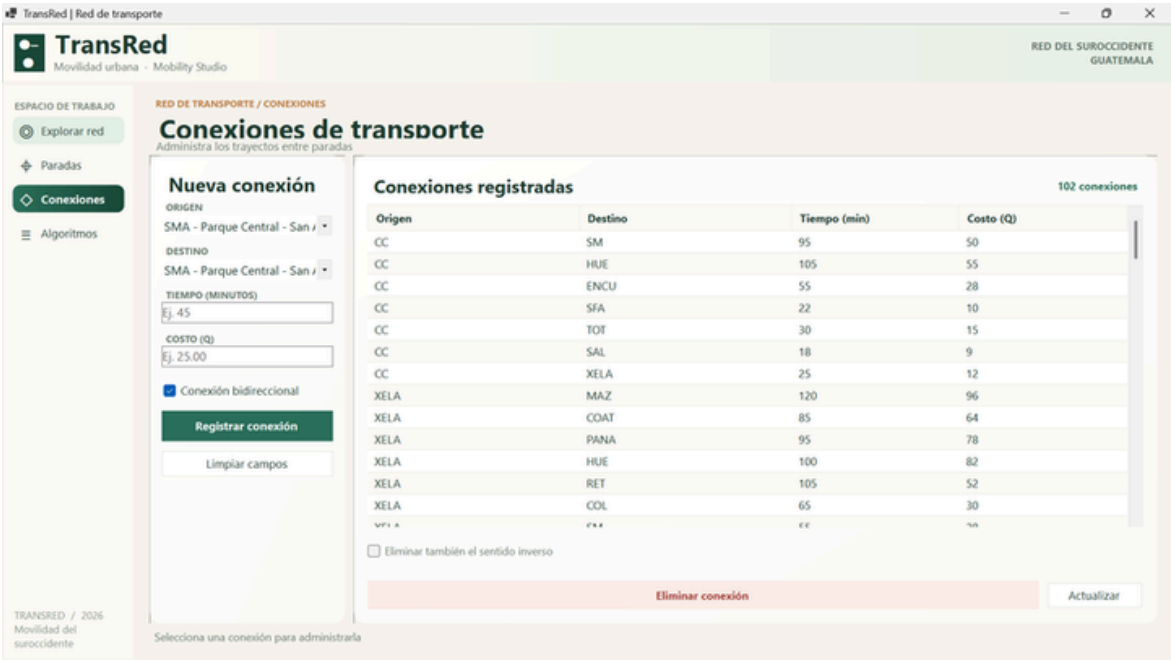


Figura E3. Conexiones y pesos.

EVIDENCIA 04

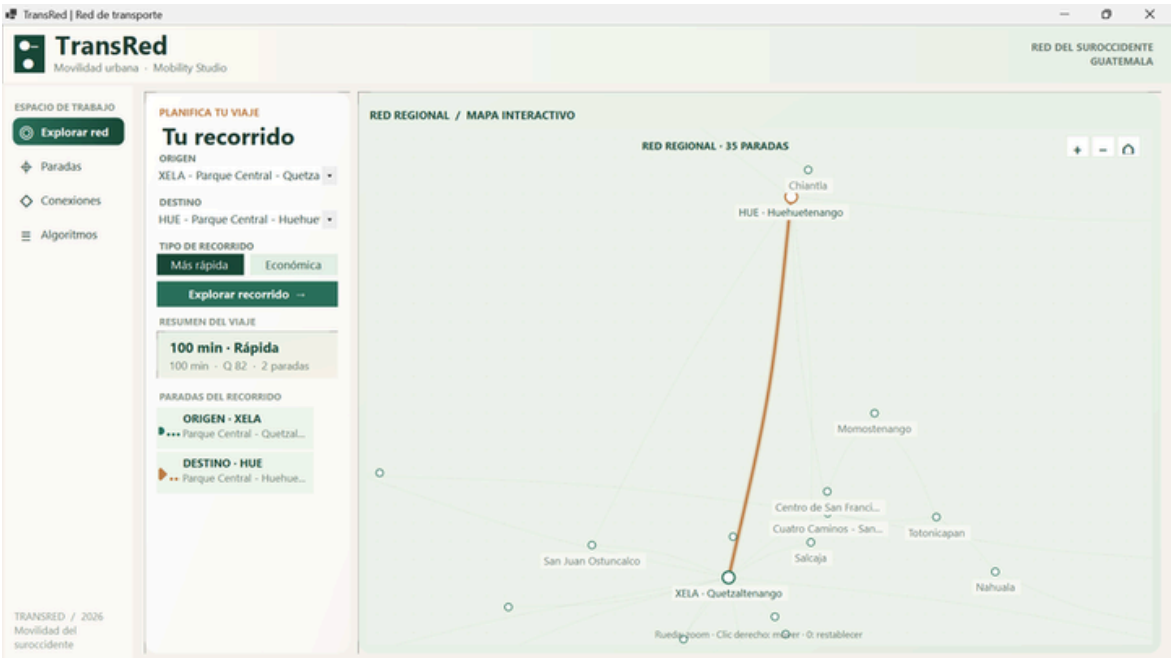


Figura E4.1 Dijkstra: criterio rápido

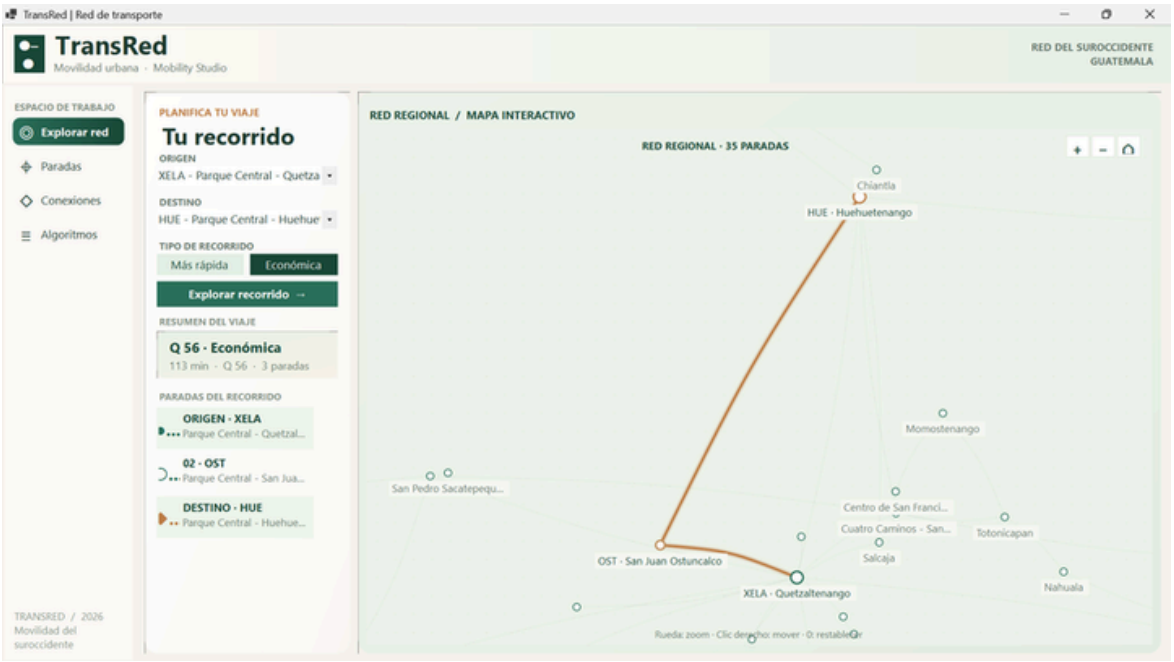


Figura E4.2. Dijkstra: criterio económico

EVIDENCIA 05



Figura E5. BFS: recorrido en anchura.

EVIDENCIA 06



Figura E6. DFS: recorrido en profundidad

EVIDENCIA 07

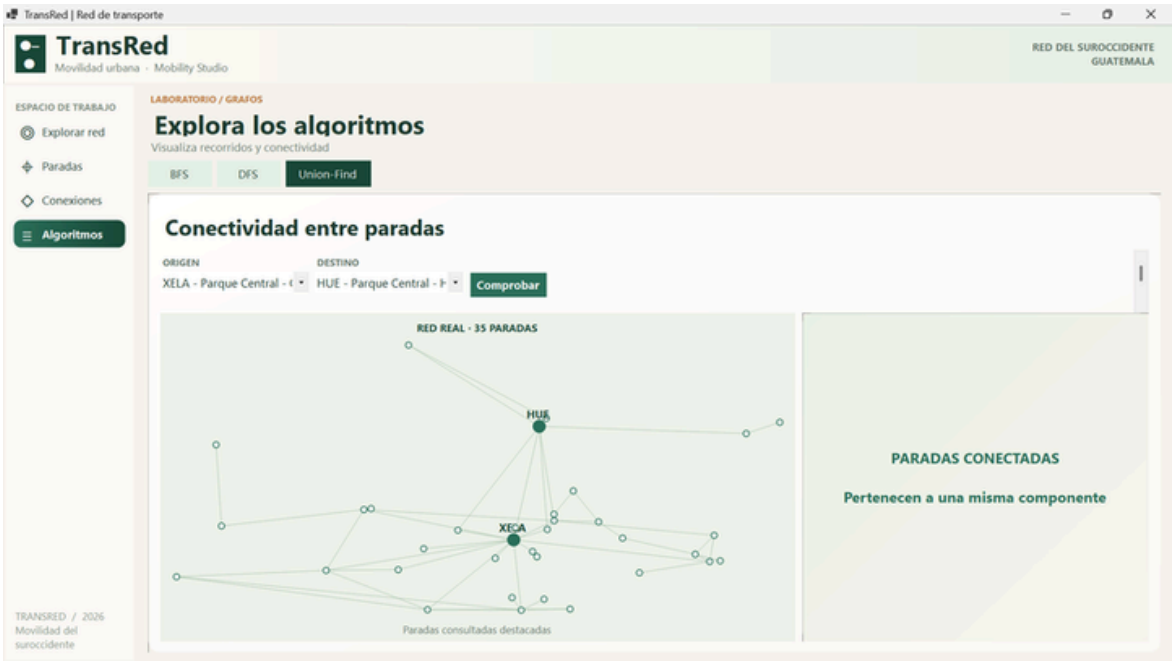


Figura E7. Union-Find: conectividad.